

Pipes

Part I

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Introduction

- Pipes are a fundamental Inter-Process Communication (IPC) mechanism in operating systems
- Used to establish a **one-way communication channel** between processes
 - A stream of bytes flows into it at one end and comes out in the same order at the other end
- File descriptors are integral to understanding how pipes work
 - When a pipe is created, two file descriptors are returned: one for reading and one for writing

I/O in Linux

- **UNIX-like operating systems strive to provide a uniform interface for interacting with system resources**
- **Many IPC mechanisms integrate with the file system abstraction, reflecting the design principle that “everything is a file”**
- **Understanding this common interface is essential for building efficient and flexible communication mechanisms between processes**

I/O in Linux

- In C, it is usual to use the `stdio` library for I/O
 - `printf()`, `scanf()`, `fopen()`, ...
- It provides a higher level of abstraction
 - Buffering, I/O converting, formatting, ...
- However, it is not always desirable/possible to use these functionalities
 - E.g., if intending to control the specific operations of a particular device, control buffer sizes, or non-blocking I/O

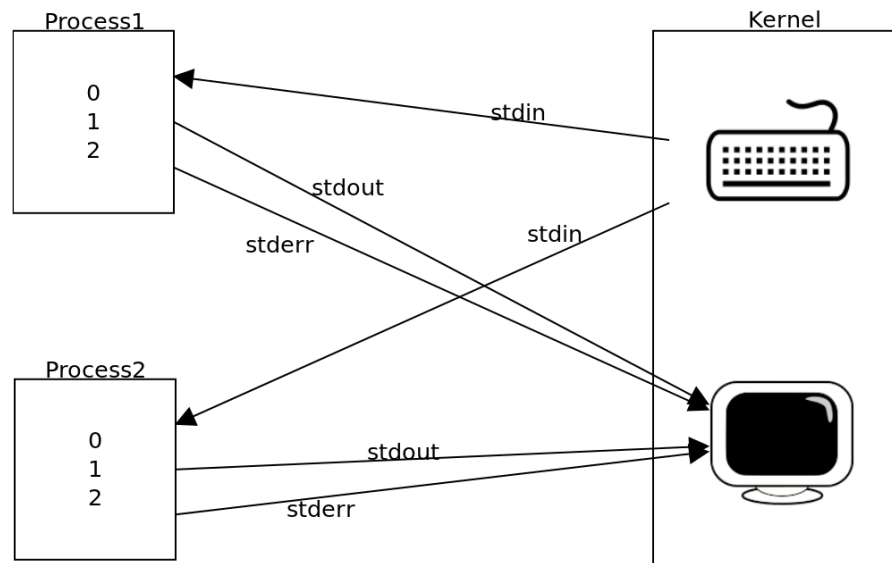
I/O in Linux

- In POSIX systems, a **file descriptor** is an integer that identifies an open file (or resource) in the system
- File descriptors provide a low-level interface for I/O operations
 - `open()`, `read()`, `write()`, `close()`, ...
- This descriptor is an index in a table, named the **file descriptors table**
 - The table is only accessible by the OS kernel and cannot be manipulated by user-level processes

File descriptor table

- Each process has its own descriptors table

- Descriptors 0, 1 and 2 are predefined: `stdin`, `stdout`, and `stderr`, respectively

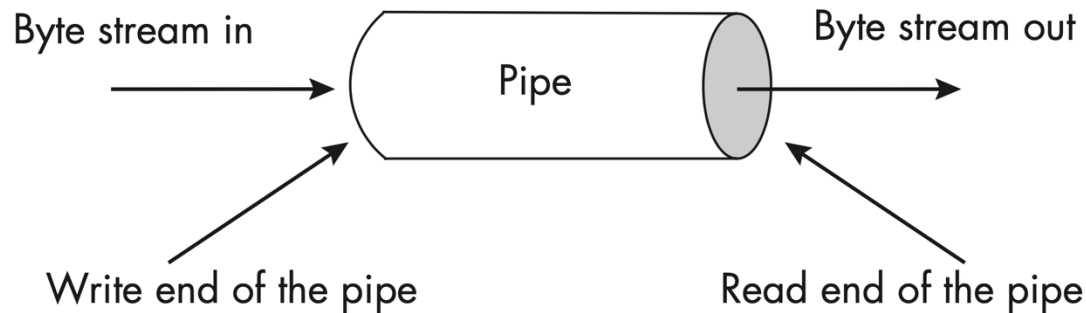


File descriptor table

- A process may open the same file/resource more than once, getting different descriptors
- If two different processes open the same file, there is no relation between the descriptors in the two processes
 - i.e., their values can be different
- File descriptors are copied in a `fork()` and preserved (usually) in an `exec` function
 - Unless they are marked with the `FD_CLOEXEC` flag

Pipes

- A **byte-oriented** unidirectional communication mechanism accessed via read and write file descriptors



- Data can flow only **in one direction** through the pipe
 - All bytes that are written to the write end can be read from the read end
 - To have bidirectional communication, it is necessary to use two pipes

Pipes

- **Utilizes a First In, First Out (FIFO) buffer in the OS kernel for message passing**
 - Ensures a sequential and orderly exchange of data between processes
- **Pipes have limited capacity**
 - The capacity of the pipe is system dependent
- **Since the buffer resides in the kernel, the OS always intervenes**
 - Which provides several advantages in synchronizing reads and writes between processes
 - (but also introduces overhead and potential performance bottlenecks, which will be addressed by using shared memory latter in the semester)

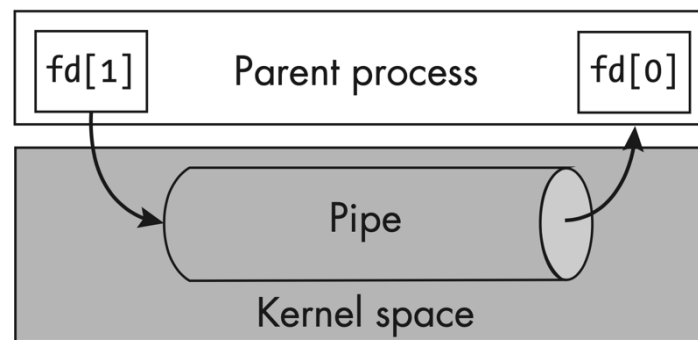
Named vs unnamed pipes

- **Unnamed pipes are used for communication between parent and child processes**
 - Exist only during the lifetime of the processes involved
 - No name in the file system
 - Data is lost once both ends are closed
 - (this will be the variant used in SCOMP)

- **Named pipes (aka FIFOs) can be used for communication between unrelated processes**
 - Persist until explicitly removed, even after the processes close the pipe
 - Have a name in the file system
 - Allows for more flexible inter-process communication

Unnamed pipes

- Communication via inherited file descriptors requires a parent–child relationship between processes
 - When `fork()` is called, the child receives a copy of the parent's file descriptor table
- Therefore, file descriptors intended for inter-process communication must be created **before** the `fork()` call



The `pipe` function

```
#include <unistd.h>

int pipe(int fd[2]);
```

- **Creates a pipe, placing two file descriptors in `fd[ ]`**

- `fd[0]` is the read descriptor
- `fd[1]` is the write descriptor

- **Returns**

- 0, upon success
- -1, in case of error

Pipe data flow

- A flow of bytes written to the pipe using the write descriptor can be subsequently read using the read descriptor
 - Pipes preserve the order of the data written into them
- Data transfers to the pipe are **atomic up to a certain limit**, defined by the constant `PIPE_BUF`
 - This ensures that writes of data smaller than or equal to `PIPE_BUF` are completed as a single, uninterrupted operation
 - Writes larger than `PIPE_BUF` may be interleaved
- Reading **removes** the data from the pipe
 - Therefore, when multiple readers read from the same pipe, no two read the same data

The `read` function

```
#include <unistd.h>

ssize_t read(int fd, void *buffer, size_t n_bytes);
```

- Attempts to read `n_bytes`, from descriptor `fd`, placing the bytes in `buffer`
- Returns
 - The number of bytes effectively read
 - 0, if there will be no more bytes to read
 - -1, in case of error

The `write` function

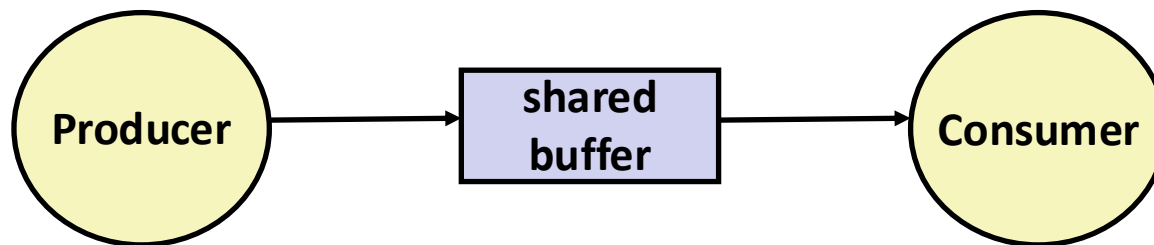
```
#include <unistd.h>

ssize_t write(int fd, void *buffer, size_t n_bytes);
```

- Attempts to write up to `n_bytes` from `buffer` to the resource referred to by `fd`
- Returns
 - The number of bytes effectively written
 - -1, in case of error

Reading and writing in pipes

- Reading/writing **synchronization** is guaranteed
 - `write()` **blocks** when the pipe is **full**
 - `read()` **blocks** if there is **no data** in the pipe
- Ideal for the bounded-buffer producer-consumer problem
 - One of the small collection of standard, well-known problems in concurrent programming (more on this in future lectures)



The `close` function

```
#include <unistd.h>

int close(int fd);
```

■ Closes the resource referred to by `fd`

- Removes the descriptor from the table
- The entry can be reused

■ Returns

- 0, in case of success
- -1, in case of error

Closing descriptors

- Pipes provide blocking behavior and bounded buffering, but do not eliminate the need for careful protocol design
- The fact that a pipe is unidirectional has an important implication
 - Every process that has a copy of the pipe's open file descriptors must **close the end of the pipe it is not going to use**
- Because of the way that read and write operations work:
 - If a consumer process reads from the pipe but has the write end open, it will block permanently when it tries to read and the pipe is empty
 - This is also why the producer process must close the write end when it finishes writing; otherwise, the consumer will block when it tries to read from the pipe

Closing descriptors

- **Closing descriptors used in a pipe indicates the end of communication by the process**
 - Write to a pipe with all read descriptors closed (no one is reading) fails and the process receives the `SIGPIPE` signal
 - Reading from a pipe with all the write descriptors closed (no one can write) returns 0 (end-of-file)
- **It is a very common and hard-to-debug mistake to forget to close the unused file descriptor on a pipe**
 - Always make sure that a process intended to read from a pipe closes the write end of the pipe before anything else

Example

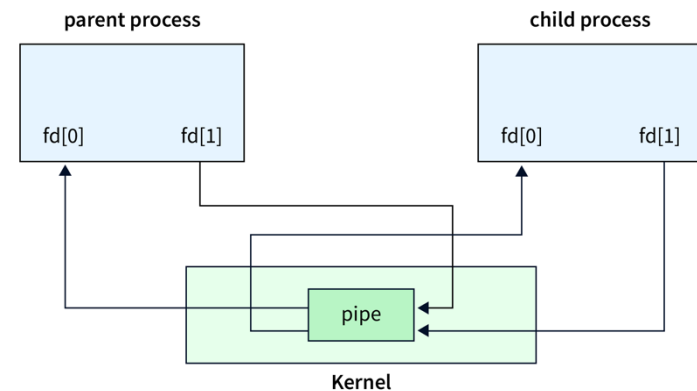
```
#define BUFFER_SIZE 80

int main(void){
    char read_msg[BUFFER_SIZE];
    char write_msg[BUFFER_SIZE] = "It works!";
    int fd[2];
    pid_t pid;
    int n;

    /* Creates a pipe */
    if(pipe(fd) == -1){
        perror("Pipe failed");
        exit(1);
    }

    /* Child inherits file descriptor table */
    pid = fork();

    ...
}
```



Example

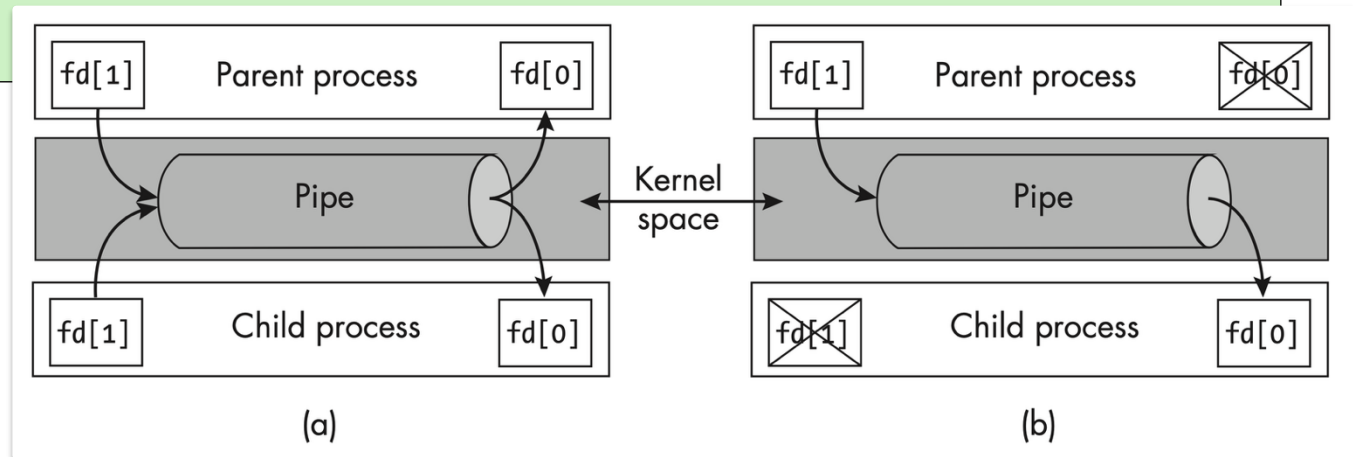
```

if(pid > 0){
    /* closes unused read descriptor */
    close(fd[0]) ;
    ...
}
else{
    /* closes unused write descriptor */
    close(fd[1]) ;
    ...
}

```

/* parent will be the producer */

/* child will be the consumer */



- (a) The state of the pipe before parent and child close their respective read and write descriptors
- (b) Its state after

Example

```
if(pid > 0){                                     /* parent will be the producer */
    /* closes unused read descriptor */
    close(fd[0]);

    /* writes string in pipe */
    n = write(fd[1], write_msg, strlen(write_msg)+1);

    /* closes write descriptor */
    close(fd[1]);

    wait(NULL);
}
else{                                           /* child will be the consumer */
    /* closes unused write descriptor */
    close(fd[1]);

    /* read string from pipe */
    n = read(fd[0], read_msg, BUFFER_SIZE);

    printf("Child read from pipe: %s\n", read_msg);

    /* closes read descriptor */
    close(fd[0]);
    exit(0);
}
```

Read/write synchronization

- **The next example demonstrates that the OS handles read/write synchronization**
 - Ensuring that each message is correctly transmitted through the pipe without interference

- **Why is no additional synchronization mechanism needed?**
 - The kernel manages concurrent writes to a pipe
 - If messages are smaller than `PIPE_BUF` (typically 4096 bytes), each `write()` call is atomic
 - The order of messages remains non-deterministic, but no race conditions occur (for writes $\leq \text{PIPE_BUF}$)

Example – Read/write synchronization

```
#define MESSAGE_SIZE 50    /* Small enough to be written atomically */
#define NUM_CHILDREN 2     /* Number of child processes */

int main(){
    int fd[2], len, bytesRead;
    pid_t pids[NUM_CHILDREN];
    char buffer[MESSAGE_SIZE];

    /* Create the pipe */
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    /* Fork multiple child processes */
    for (int i = 0; i < NUM_CHILDREN; i++) {
        if ((pids[i] = fork()) == 0) {
            close(fd[0]);
            len = snprintf(buffer, MESSAGE_SIZE, "My PID is %d", getpid());
            write(fd[1], buffer, len);
            close(fd[1]);
            exit(EXIT_SUCCESS);
        }
    }
    ...
}
```


Example – Read/write synchronization

```
/* parent closes unused write descriptor */
close(fd[1]);

/* Read messages from the pipe (order is non-deterministic) */
for(int i = 0; i < NUM_CHILDREN; i++) {
    bytesRead = read(fd[0], buffer, MESSAGE_SIZE - 1);
    if(bytesRead > 0) {
        buffer[bytesRead] = '\0';
        printf("Parent received: %s", buffer);
    }
}

/* closes read descriptor */
close(fd[0]);

for(i = 0; i < NUM_CHILDREN; i++){
    wait(NULL);
}

return 0;
}
```

Exercise 1

- Implement a program that creates a child process
- Parent sends two integers (provided by user) to the child using a pipe
- Child reads the integers, calculates their sum and terminates with the sum as its exit value
 - Recall that exit status is limited to 8 bits (0–255)
- Parent waits for the child to terminate and prints result

Exercise 2

- **Change the previous exercise to support an unknown number of integers, sent from parent to child**
- **Parent continuously reads an integer (provided by user) and sends it to the child using a pipe**
 - Assume that it stops sending values when the user enters the value 0
- **Child reads all the integers, calculating their sum and terminating with the sum as its exit value**
- **Parent waits for the child to terminate and prints result**